Documentação Técnica: Simulador DRIMA

(Dynamics of Randomly Interacting Moving Agents)

Igor Benites Moura[1], Gabriel Gonzaga Chung[1], and Mateus Kenzo Iochimoto[1]

[1]Faculdade de Computação e Informática (FCI) , Universidade Presbiteriana Mackenzie , São Paulo, SP – Brasil

17 de novembro de 2025

Conteúdo

1 Visão Geral do Projeto **3**
1.1 Componentes Principais 3
1.2 Tecnologias Utilizadas 3

2 Guia de Instalação e Execução **3**
2.1 Pré-requisitos 3
2.2 Instalação 3
2.3 Estrutura de Arquivos 4
2.4 Execução 4

3 Manual do Usuário (Interface Gráfica) **4**
3.1 Painel Funcional (Visualização) 4
3.2 Painel de Controle (Configuração) 5
3.3 Janela: Add New Agent (Pop-up) 6
3.4 Janela: Advanced Graph Filter (Pop-up) 7

4 Arquitetura do Software (Backend) **8**

1 Visão Geral do Projeto

Nome do Projeto: Simulador DRIMA (Dynamics of Randomly Interacting Moving Agents)

Objetivo: O Simulador DRIMA é uma aplicação de software desenvolvida para modelar, executar e analisar simulações de sistemas multiagente. O foco principal é estudar a **evolução do comportamento** e a **propagação de influência** dentro de uma população de agentes com regras dinâmicas.

1.1 Componentes Principais

- **Agentes Determinísticos (D-Agents):** Agentes "teimosos"ou "influenciadores"com um padrão de comportamento fixo que não muda.
- **Agentes Randômicos (R-Agents):** Agentes "influenciáveis"ou "indecisos". Seus padrões de movimento são uma distribuição de probabilidade que é dinamicamente alterada pela influência dos agentes vizinhos.

1.2 Tecnologias Utilizadas

- **Linguagem:** Python 3.10.x
- **Interface Gráfica (GUI):** CustomTkinter
- **Visualização de Dados:** Matplotlib
- **Formato de Dados:** JSON
- **Compressão:** Zipfile

—

2 Guia de Instalação e Execução

2.1 Pré-requisitos

- Python 3.10 ou mais recente.
- Gerenciador de pacotes `pip`.

2.2 Instalação

Passo 1 — Obter os arquivos do projeto

Obtenha a pasta do projeto (ex: `DRIMA_py-final-version`) e navegue até ela pelo terminal.

```
cd C:\Caminho\Para\DRIMA_py-final-version
```

Passo 2 — Instalar dependências

O projeto contém um arquivo `requirements.txt` com todas as bibliotecas necessárias.

```
pip install -r requirements.txt
```

2.3 Estrutura de Arquivos

O projeto **deve** seguir a estrutura de pastas abaixo para que os `imports` do Python funcionem:

```
/DRIMA_py-final-version/  <-- Pasta raiz
|
|-- main.py                 (Script principal para executar)
|-- drima.json              (Arquivo de configuracao)
|-- requirements.txt        (Lista de dependencias)
|-- ... (outros arquivos de config, .md, etc.)
|
+-- project/                (Pasta "project")
|   |-- __init__.py         (Arquivo vazio)
|   |-- Representacao.py
|   |-- AgentSpawner.py
|   |-- ... (etc.)
|
+-- imagens/                (Pasta para imagens da doc)
```

2.4 Execução

1. Certifique-se de estar na pasta raiz do projeto (`DRIMA_py-final-version`).
2. Execute o arquivo `main.py`:

No terminal, execute:

```
python main.py
```

A interface gráfica do simulador será iniciada, como mostrado na Figura 1.

—

3 Manual do Usuário (Interface Gráfica)

A interface é dividida em dois painéis principais: o **Painel Funcional** (esquerda, Figura 2) e o **Painel de Controle** (direita, Figura 3).

3.1 Painel Funcional (Visualização)

Esta área exibe os resultados da simulação.

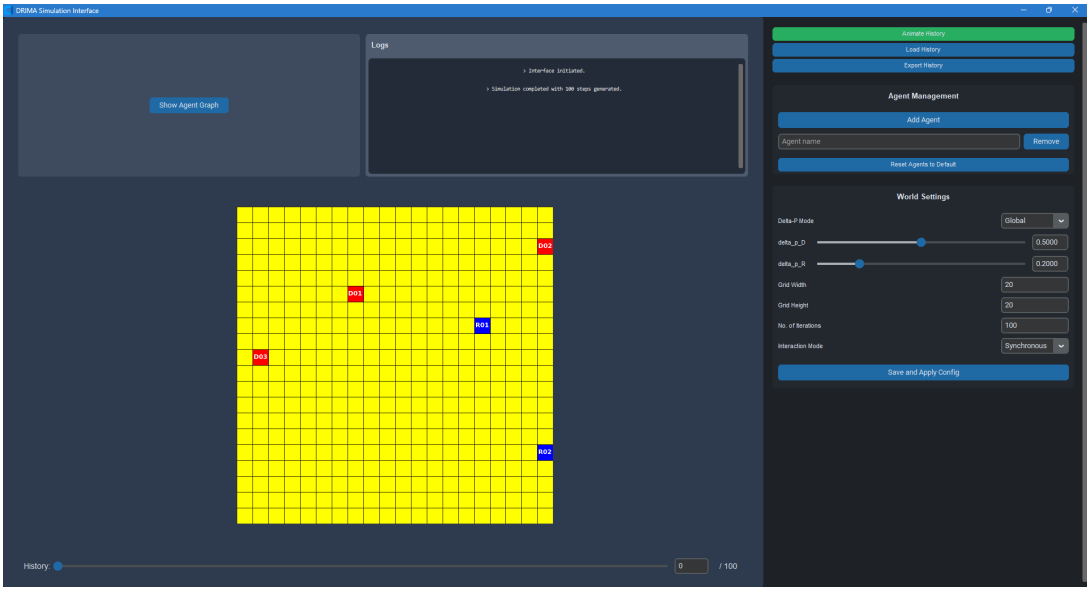


Figura 1: A interface principal do Simulador DRIMA após a inicialização.

- **Simulation Visualization:** O grid principal (fundo amarelo) onde o estado atual da simulação é exibido. Agentes 'D' são **vermelhos** e agentes 'R' são **azuis**.
- **History Control (Barra inferior):**
  - **Slider e Caixa de Texto:** Permitem navegar ou pular para uma iteração específica da simulação.
  - **Label / `X`:** Mostra o número total de iterações na história.
- **Show Agent Graph (Botão):**
  - Abre a janela "Advanced Graph Filter"para análise detalhada de um agente.
- **Logs:** Um console que exibe o status do sistema, confirmações de ações (ex: "Agent added") e mensagens de erro.

3.2 Painel de Controle (Configuração)

Esta área é usada para configurar, executar e gerenciar a simulação.

- **Controles Principais:**
  - **Animate History:** Inicia/para uma "reprodução"da simulação a partir do ponto atual do slider.
  - **Load World (.zip):** Abre um explorador de arquivos para carregar um estado de simulação (`.zip`) salvo anteriormente.
  - **Export World (.zip):** Salva a configuração atual e a história completa da simulação em um arquivo `.zip` comprimido.
- **Agent Management:**
  - **Add Agent:** Abre a janela "Add New Agent"(Figura 4).



Figura 2: O Painel Funcional (esquerda), exibindo a grade de simulação, o gráfico de agente e os logs.

  - **Caixa de Texto (Agent name) e Botão Remove:** Remove o agente especificado do estado inicial.
  - **Reset Agents to Default:** Descarta todos os agentes atuais e os substitui por um novo conjunto de 5 agentes aleatórios.

- **World Configuration:**
  - **Sliders (`delta_p_D`, `delta_p_R`):** Controla a "força de influência"dos agentes. O valor pode ser ajustado pelo slider ou digitado na caixa.
  - **Campos de Texto:** `Grid Width`, `Grid Height`, `No. of Iterations`.
  - **Dropdown (`Interaction Mode`):** `Synchronous` (padrão) ou `Asynchronous`.
  - **Save & Apply Config: Botão principal.** Salva as configurações deste painel no `drima.json` e **reinicia e recalcula a simulação** com os agentes atuais e os novos parâmetros.

3.3 Janela: Add New Agent (Pop-up)

Conforme visto na Figura 4, esta janela permite a criação detalhada de agentes.

- **Agent Type:** Escolha entre "Random (R)"(influenciável) ou "Deterministic (D)"(teimoso).
- **Position (X, Y):** Opcional. Se vazio, a posição é aleatória.

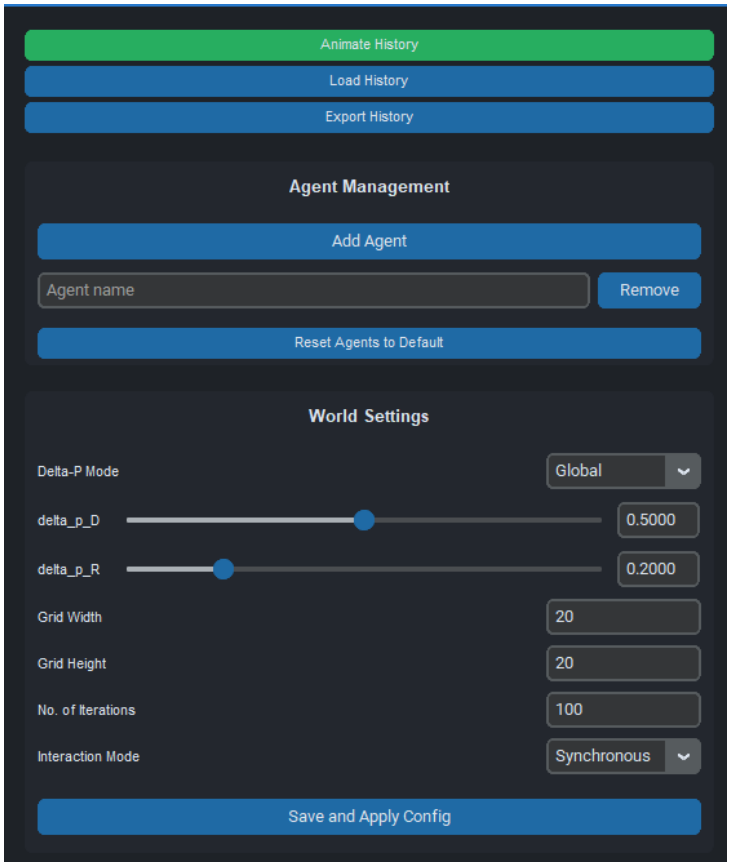


Figura 3: O Painel de Controle (direita), com as seções de Gerenciamento de Agentes e Configuração do Mundo.

- **Actions by Direction:** A "rosa dos ventos"para definir a probabilidade de movimento para cada uma das 9 direções (L = Leste, X = Ficar Parado).
- **Stay Still (Checkbox):** Um atalho que define a probabilidade de "X"para 1.0 e as outras para 0.0.
- **Validação:** Ao clicar em "Add", o sistema verifica se a soma é 1.0. Se for ligeiramente menor (ex: 0.9999), ele pergunta se o usuário deseja **normalizar** os valores para 1.0.

3.4 Janela: Advanced Graph Filter (Pop-up)

Esta janela (Figura 5) é usada para configurar a visualização de dados.

- **Agent Name:** Nome do agente a ser analisado (ex: "R01").
- **History Range:** Permite filtrar um intervalo de tempo (ex: do frame 1000 ao 2000).
- **Directions to Display:** Permite ligar ou desligar a visualização de direções específicas (NO, N, NE, O, X, L, SO, S, SE).

—

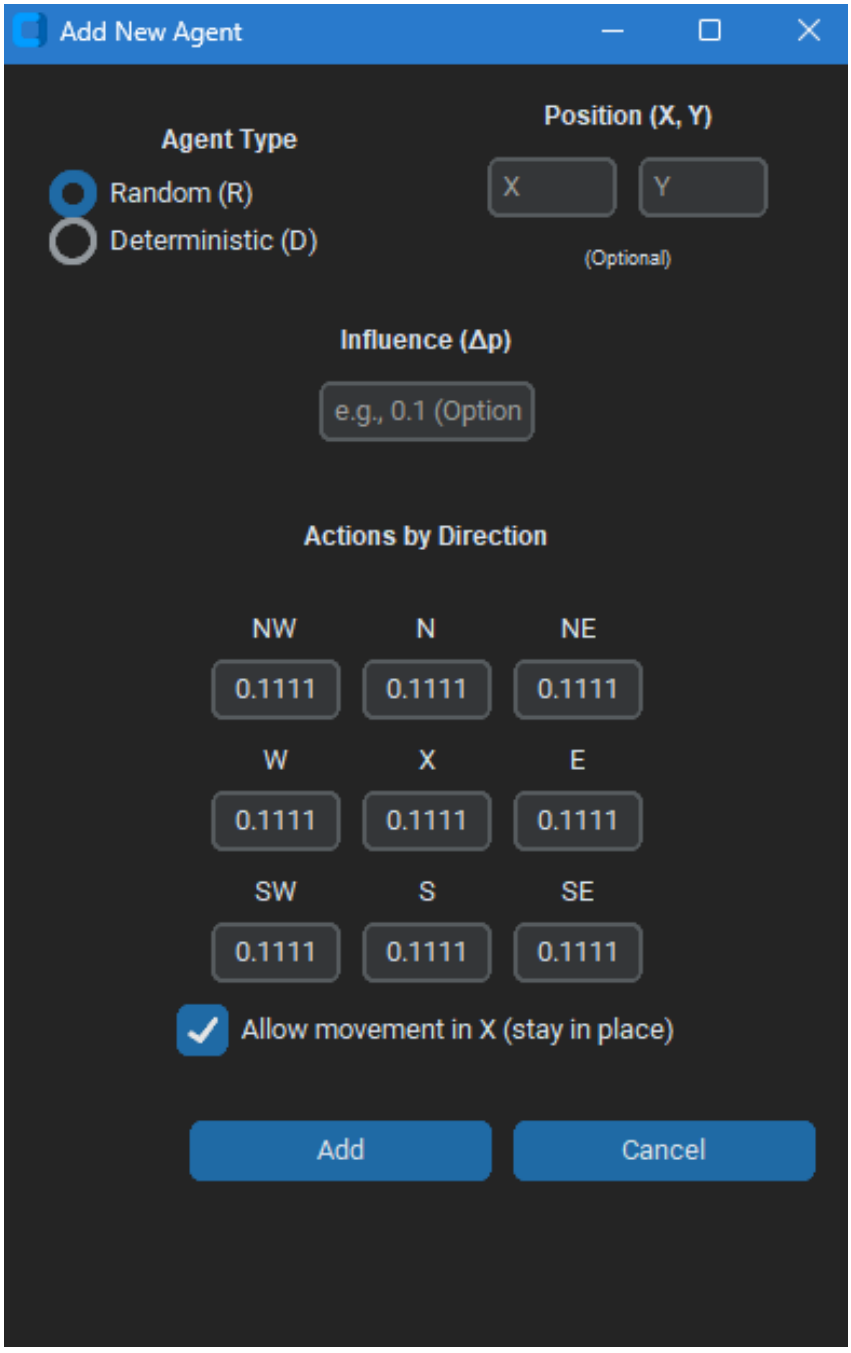


Figura 4: A janela "Add New Agent"com a "rosa dos ventos"para configuração do padrão de movimento.

4 Arquitetura do Software (Backend)

Os arquivos na pasta `project/` contêm a lógica principal da simulação.

- `Representacao.py`: Define as classes de dados `Agent` e `World`. É o "molde"que armazena os dados da simulação.
- `AgentSpawner.py`: Contém a lógica para criar novos agentes, incluindo o método `_get_next_name` que garante nomes sequenciais e únicos (ex: "R05"após "R04").
- `Interacoes.py`: **O coração do DRIMA.** Define como um agente 'R' altera seu

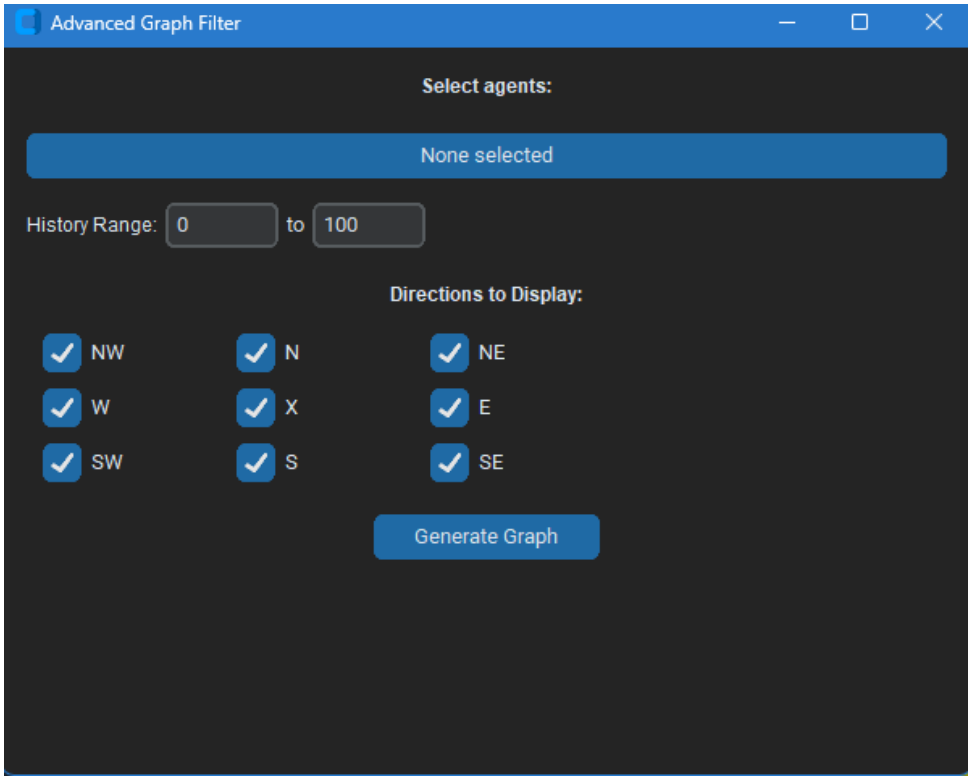


Figura 5: A janela "Advanced Graph Filter", permitindo a seleção de agente, intervalo da história e direções.

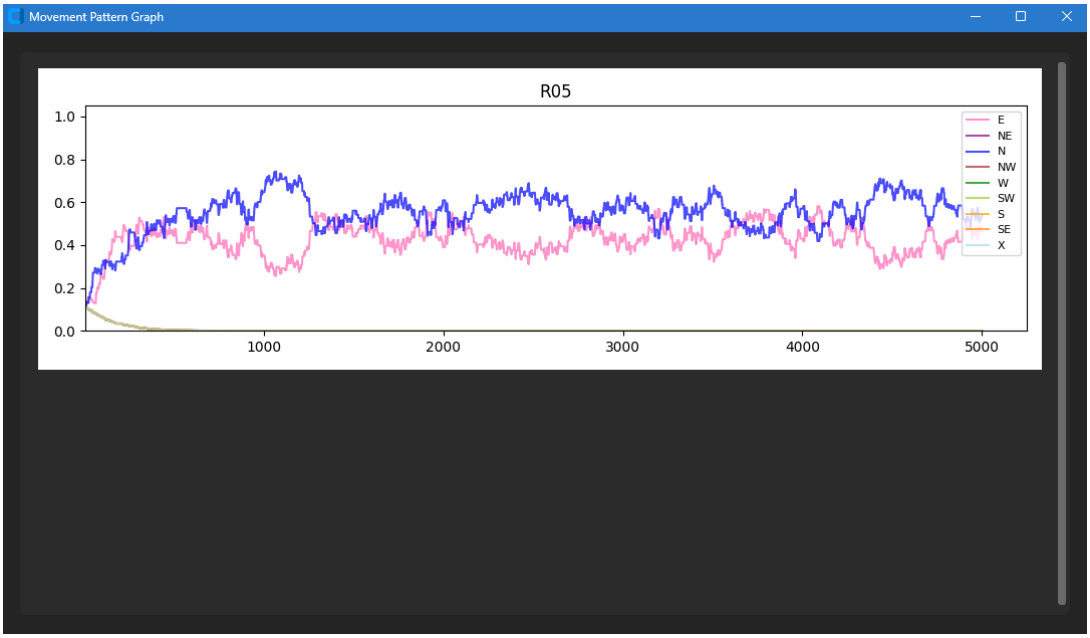


Figura 6: Exemplo de um gráfico de saída. Neste caso, é exibido um cenário de **não-convergência**, onde o agente 'R05' oscila entre duas opiniões (linhas rosa e azul) sem nunca se decidir por uma.

`padrao_movimento` para se assemelhar a um vizinho, com base no `delta_p`.

- `Movimento.py`: Lógica que move um agente com base em uma escolha ponderada

de seu `padrao_movimento`.

- `WorldUpdater.py`: O motor da simulação. Orquestra a ordem de interação e movimento para cada passo da história.
- `GraphVisualizer.py`: Módulo de análise. Contém a lógica do Matplotlib para `show_filtered_graph` que gera o gráfico de análise.
- `FileHandler.py`: Utilitário para salvar/carregar o `drima.json` e para comprimir/descomprimir os arquivos `.zip`.